

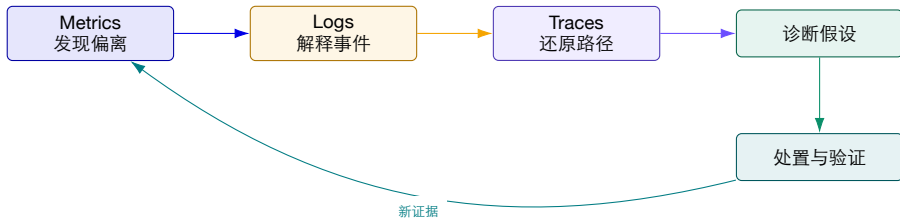
面向 AI 原生系统的智能运维

第 4 讲 | 智能监控与可观测性工程 (2/3)

结构化日志、分布式追踪与跨信号故障诊断

陈鹏飞 | 中山大学

与上一讲的连接：有了信号，还要把证据组织起来



第3讲 建立指标、OpenTelemetry 与存储系统。

第4讲 深入日志、Trace、采样、关联和诊断实验。

第5讲 扩展到 GPU、Token、模型质量与成本。

本讲学习目标

1. 把应用输出设计为具有稳定语义、可关联、可治理的结构化日志；
2. 解释 ELK 与 Loki 的数据模型、索引方式、成本差异和适用边界；
3. 使用 Trace、Span、上下文传播和语义约定描述端到端请求；
4. 区分头部采样、尾部采样、回溯采样与全请求近似保留；
5. 理解 Jaeger v2、OpenTelemetry Collector 与存储后端的职责；
6. 联合指标、日志、Trace、拓扑和变更形成可复查的诊断证据链；
7. 完成一个三服务推理系统的日志—Trace 关联实验。

目标不是“装好 ELK/Jaeger”，而是能解释一条证据怎样产生、怎样丢失、怎样误导诊断。

开场事故：同一个慢请求，三个系统给出三种解释

业务现象

- ▶ RAG 智能客服 P95 从 2.1s 上升到 9.4s；
- ▶ 只有部分租户受影响，错误率没有明显上升；
- ▶ 值班人员在日志平台、指标平台和 Trace 平台之间切换；
- ▶ 28 分钟后才确认向量数据库连接池耗尽。

三个孤立视角

- ▶ 指标：检索服务延迟升高，但不知道哪些请求；
- ▶ 日志：大量 timeout，但缺少统一字段和调用上下文；
- ▶ Trace：部分慢请求被采样丢弃，剩余请求看似正常；
- ▶ 变更记录：连接池参数在 12 分钟前被调整。

事故教训 诊断失败往往不是“完全没有数据”，而是数据缺少共同身份、时间语义和保留策略。

课堂推理： 四段证据能否属于同一个请求？

<code>metric</code>	retrieval latency P95 = 6.8s	<code>trace</code>	gateway → rag → vector-db
<code>log</code>	pool timeout after 5000 ms	<code>resource</code>	pod=rag-7f9, node=gpu-worker-2
<code>change</code>	max_connections: 80 → 20	<code>time</code>	12:03:18.142Z / clock offset unknown

1. 哪些字段能证明它们来自同一个请求、服务实例和时间窗？
2. 如果 **Trace** 没有被保留，日志还能否重建调用路径？
3. 如果变更时间来自本地时钟，因果先后是否可信？

本讲主线 让“看起来相关”升级为“可以被验证的关联”。

Trace 算例：并行分支为什么不能直接相加？

网关同时发起检索 A 与策略检查 B，等待两者完成后调用模型 C；忽略网络开销，所有区间共用时钟。

Span	执行区间（ms）	耗时	依赖
A：检索	0-300	300 ms	与 B 并行
B：策略检查	0-100	100 ms	与 A 并行
C：模型	300-800	500 ms	等待 A、B 都完成
网关总时间	0-800	800 ms	不是 300+100+500

$$T = \max(T_A, T_B) + T_C = 800\ ms$$

推导与判断 将 B 加速到 10 ms，总耗时仍是 800 ms；将 A 加速到 200 ms，则总耗时变为 700 ms。
优化应沿依赖图上的关键路径推进。

算例使用假设数据。

本讲路线图

1. 日志工程：事件、结构、上下文、质量与隐私；
2. 日志平台：ELK、Loki、OTLP、查询和保留；
3. 分布式追踪：Span、传播、OpenTelemetry、Jaeger 与采样；
4. 研究案例：SwissLog、TraStrainer、Mint、ZeroTracer、AgentTracer；
5. 跨信号关联：Exemplar、统一时间、对象身份与证据模型；
6. 实验：搭建三服务推理栈，注入慢调用并完成诊断。

日志语义 → 请求路径 → 采样存储 → 关联诊断

一、把日志变成工程证据

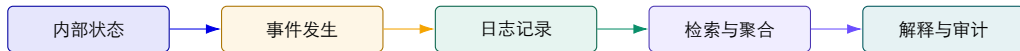
日志不是“打印一些文字”，而是记录系统状态变化与决策上下文的数据产品。

日志最擅长回答：某个时刻发生了什么事件？

状态变化 模型加载、连接建立、任务开始、缓存失效、配置生效。

决策依据 选择哪个模型、路由到哪个后端、为什么重试或降级。

失败细节 异常类型、错误码、堆栈、输入摘要和恢复动作。



指标压缩了大量事件；Trace 组织了请求路径；日志保留事件的局部语义和细节。

事件、日志级别与动作必须分开设计

事件	建议级别	说明	预期动作
GPU 驱动崩溃	CRITICAL	当前工作负载不可继续	故障转移、Page 值班
模型加载失败	ERROR	一次操作失败，可能影响请求	检查版本、存储与显存
重试第 2 次	WARN	尚未失败，但风险上升	聚合重试率与下游状态
模型版本切换	INFO	正常且值得审计的状态变化	关联变更与效果指标
张量形状详情	DEBUG	仅用于短期诊断	默认关闭或按需开启

设计原则 级别表示处置严重度，不应被当作“信息多少”的开关；可查询字段应独立于 message。

“JSON 日志” 不一定是结构化日志

只是 JSON 编码

```
{"msg":"user 42 failed",  
  "data":"latency=8s",  
  "extra":"prod" }
```

- ▶ 字段类型与含义不稳定;
- ▶ 关键实体仍埋在字符串中;
- ▶ 不同服务无法统一查询。

稳定结构与语义

```
{"event.name":"inference.failed",  
  "user.id_hash":"...",  
  "duration_ms":8120,  
  "deployment.environment":"prod"  
}
```

- ▶ 字段名、类型、单位和隐私策略明确;
- ▶ 下游可以验证、聚合和关联;
- ▶ Schema 可以版本化和测试。

一条可用于诊断的推理日志应该包含什么？

```
{ "timestamp": "2026-08-21T12:03:18.142Z",  
  "severity_text": "ERROR", "event.name": "retrieval.timeout",  
  "service.name": "rag-api", "service.version": "2026.08.4",  
  "trace_id": "4f2c8a9e...", "span_id": "a13f09c2...",  
  "deployment.environment": "prod", "k8s.namespace.name": "ai",  
  "db.system": "milvus", "server.address": "vector-db",  
  "duration_ms": 5012, "timeout_ms": 5000,  
  "error.type": "PoolTimeout", "retry_count": 2,  
  "message": "vector retrieval timed out" }
```

谁 服务、版本、环境、
资源对象。

什么 事件、操作、结果
和错误类型。

关联 时间、Trace/Span、
依赖与重试。

核心字段分为五类：不要把所有信息塞进 message

`timestamp / observed_timestamp` 事件时间与采集时间

`service.name / version` 服务身份与版本

`trace_id / span_id` 请求上下文

`event.name / outcome` 稳定事件与结果

`error.type / stacktrace` 失败语义

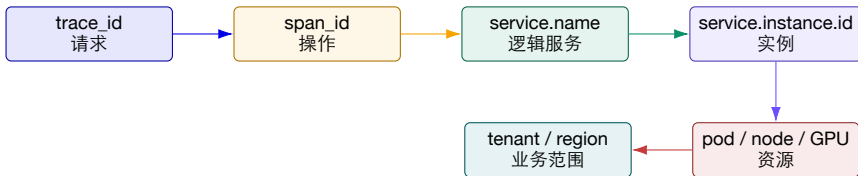
`k8s.* / cloud.*` 资源身份

`duration_ms / bytes` 带单位的数值

`message` 面向人的简短说明

Schema 策略 优先采用 OpenTelemetry Semantic Conventions 或 ECS；自定义字段应有命名空间、类型、单位和所有者。

关联上下文：一次请求需要同时绑定逻辑身份与资源身份



逻辑关联 一个用户任务、一次 Agent Run、一次模型请求包含哪些步骤。

物理关联 这些步骤落在哪个 Pod、节点、GPU、驱动与网络路径。

没有双重身份，故障只能停留在“服务慢”或“机器异常”，难以建立请求—资源因果链。

堆栈与多行日志：人类可读不等于管道可处理

常见问题

- ▶ Python/Java 堆栈被拆成几十条独立日志；
- ▶ 第一行带时间，后续行没有上下文；
- ▶ Collector 重启后多行边界识别错误；
- ▶ 超长堆栈触发单条记录大小限制。

处理策略

- ▶ 源端优先输出结构化 exception 字段；
- ▶ 文件采集配置 multiline 规则与超时；
- ▶ 保存 error.type、摘要和完整堆栈；
- ▶ 对重复堆栈做指纹与聚合，但保留样本。

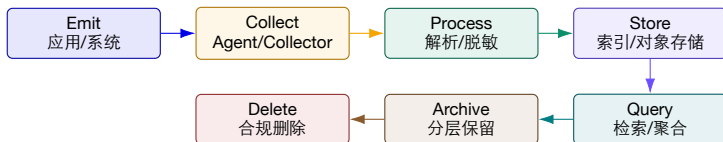
边界 不要为了节省成本只保留错误摘要；没有完整堆栈时，代码位置与调用上下文可能无法恢复。

六个常见日志反模式

1. 字符串拼接一切：字段不可聚合；
2. 成功路径有日志，失败路径没有；
3. 每次循环 **INFO**：制造噪声与成本；
4. 直接记录 **Prompt/Token/PII**：泄露风险；
5. 错误被重复记录：同一异常多层打印；
6. 字段语义漂移：latency 单位或范围变化。

审查方式 对日志做像 API 一样的格式与使用方式审查：消费者是谁？字段能否稳定？发生故障时是否足够？是否泄露数据？

日志生命周期：生成、采集、处理、存储、查询、归档与删除

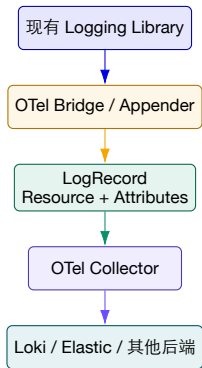


可靠性 缓冲、背压、重试、磁盘队列与丢弃指标。

治理 Schema、租户、访问、脱敏、保留与删除。

可用性 查询延迟、索引成本和冷热分层。

2026 年的 OpenTelemetry Logs：复用现有日志并统一上下文



关键认识

- ▶ Logs 已是 OTel 稳定信号；
- ▶ 自动埋点可注入 TraceId/SpanId；
- ▶ Collector 可解析遗留格式、补充资源属性；
- ▶ 统一 OTLP 不等于统一后端；
- ▶ 应优先桥接现有日志库，而非重写所有调用。

Schema 演化与数据质量：日志管道也需要测试

质量维度	失败示例	可执行检查
完整性	ERROR 缺少 trace_id 或 service.version	必填字段覆盖率、空值率
一致性	duration 有时是 ms、有时是 s	类型与单位格式与类型测试
及时性	Collector 队列积压 15 分钟	event 与 observed 时间差
唯一性	同一异常被五层重复记录	event fingerprint 与去重率
有效性	枚举中出现未知 outcome	Schema Registry / CI 校验
可关联性	Pod 名称变化但服务身份缺失	Resource 属性覆盖率

演化规则 新增可选字段通常兼容；改名、改类型、改单位或改变事件含义必须版本化，并为查询和告警提供迁移期。

隐私与成本控制要发生在进入后端之前

处理顺序

1. 识别密钥、Token、Prompt、账号和 PII;
2. 删除不必要字段;
3. 哈希/掩码需要关联但不需明文的标识;
4. 对 DEBUG、重复成功日志做采样;
5. 按级别、租户、数据域路由到不同保留层。

不可取的捷径

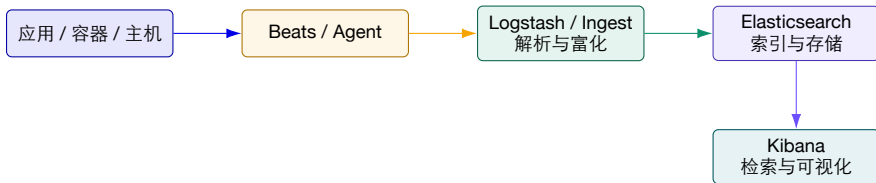
- ▶ 先把原始数据全部送到后端再脱敏;
- ▶ 把 trace_id/user_id 设为索引标签;
- ▶ 对错误日志使用与成功日志相同采样率;
- ▶ 只按日志级别决定保留期;
- ▶ 让诊断智能体绕过租户与权限边界。

原则 最小化采集不是少留证据，而是留下完成诊断、审计和合规所必需的最小充分信息。

二、日志平台：ELK、Loki 与统一采集

不同后端的核心差异在索引对象、查询路径、成本模型和运维复杂度。

ELK 数据流：把原始事件转换为可搜索文档



优势 字段索引丰富，复杂过滤、聚合和全文搜索能力强。

成本 索引、分片、映射、Merge 与副本消耗资源。

治理 必须管理 Mapping、字段爆炸、生命周期和权限。

Elasticsearch：查询快的代价是提前决定怎样索引

写入侧

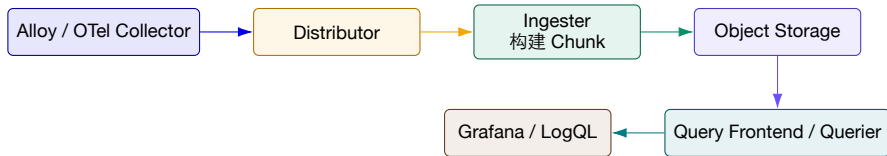
- ▶ 文档字段经过 Analyzer、类型映射和倒排索引；
- ▶ 动态 Mapping 方便，但可能造成字段爆炸；
- ▶ 分片过多产生内存和协调开销；
- ▶ Data Stream 适合仅追加的时序日志。

生命周期

- ▶ Hot：近期高频写入和查询；
- ▶ Warm：较少更新，仍需快速检索；
- ▶ Cold/Frozen：低成本保留与按需恢复；
- ▶ Delete：满足保留期和合规删除。

课堂判断 需要任意字段快速检索、复杂聚合与安全分析时，Elastic 更有优势；但要为索引和集群运维付费。

Loki: 只索引日志流标签，内容按需扫描



索引对象 service, namespace, environment 等低基数流标签。

查询方式 先选日志流，再对内容和结构化元数据过滤、解析和聚合。

Loki 的关键设计：标签用于找“哪一类日志”，元数据用于找“哪一条事件”

字段	存储为索引标签	存储为结构化元数据/日志字段
service.name	推荐：稳定、查询频繁、集合有限	可同时保留
deployment. environment	推荐：prod/staging 等有限值	可同时保留
k8s.namespace.name	通常合适	可同时保留
service.instance.id	新部署通常不推荐为标签	推荐
k8s.pod.name	易随副本变化，谨慎	推荐
trace_id / user_id	禁止：近乎每条记录唯一	推荐，按需过滤

原因 每个新的标签值组合都会创建新日志流；高基数会产生大量小 Chunk、巨大索引和低效查询。

版本提醒：Promtail 已在 2026-03-02 结束生命周期

本地材料中的经典路径

Container Logs → Promtail → Loki

- ▶ 仍适合理解文件发现、解析和标签处理；
- ▶ 旧配置可作为迁移输入；
- ▶ 不应作为 2026 年新项目的默认选型。

当前推荐路径

Logs → Grafana Alloy / OTel Collector

→ Loki OTLP

- ▶ 统一日志、指标和 Trace 管道；
- ▶ 复用 OTLP 与 Resource 属性；
- ▶ 新功能在 Alloy 等受支持客户端演进。

OTLP 日志进入 Loki: 标签与结构化元数据必须显式治理

```
exporters:
  otlphttp/loki:
    endpoint: http://loki:3100/otlp
service:
  pipelines:
    logs:
      receivers: [otlp, filelog]
      processors: [memory_limiter, batch]
      exporters: [otlphttp/loki]
```

映射规则

- ▶ Resource 属性可映射为索引标签;
- ▶ 其他 Resource/Scope/Log 属性进入结构化元数据;
- ▶ service.name 会规范化为 service_name;
- ▶ 非字符串 Body 会被字符串化;
- ▶ Loki 3.0+ 默认启用结构化元数据。

LogQL: 先缩小日志流，再解析字段和计算指标

```
{service_name="rag-api", deployment_environment="prod"}  
  | json  
  | event_name="retrieval.timeout"  
  | duration_ms > 5000  
  
sum by (service_name) (  
  count_over_time({deployment_environment="prod"}  
    | json | severity_text="ERROR" [5m])  
)
```

流选择器 尽早使用稳定
标签缩小数据范围。

解析阶段
JSON/regexp/pattern 只
处理候选日志。

指标化 日志可转成计数
与分布，但需控制查询成
本。

ELK 还是 Loki? 从查询模式和组织能力反推选型

维度	Elasticsearch / ELK	Loki
索引策略	多字段倒排与列式结构	日志流标签索引，内容按需扫描
典型优势	任意字段搜索、复杂聚合、安全分析	对象存储、Grafana 集成、成本较低
典型风险	分片、Mapping、索引和 JVM 运维复杂	高基数标签、小 Chunk、宽范围内容扫描
适合团队	有搜索平台能力与复杂检索需求	已使用 Prometheus/Grafana，查询以服务/时间为入口
混合方案	热日志/安全日志进入 Elastic	大规模应用日志进入 Loki，冷数据归档对象存储

不要只比功能表 先列出事故查询：入口字段是什么、允许多慢、保留多久、每天多少数据、谁负责运维。

日志保留不是“所有数据统一保存 30 天”

热层 | **1-7 天**
ERROR/WARN、变更窗口、完整字段；低延迟检索。

温层 | **7-30 天** INFO 降采样、重复事件聚合；支持复盘和趋势。

冷层 | **30-180 天** 对象存储压缩归档；按工单恢复，满足审计。

保留价值 = 诊断价值 + 审计价值 + 学习价值 - 存储成本 - 隐私风险

同一日志的 message、堆栈、结构化字段和聚合摘要可以有不同保留期；删除策略必须可验证。

SwissLog 1/5: 并发系统的日志为什么天然“交错”?

现实输入

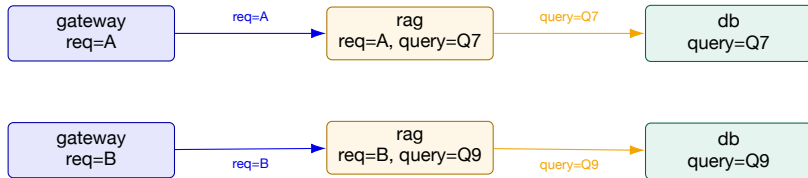
- ▶ 多线程、多实例、多个微服务并行输出;
- ▶ 同一请求的日志被其他请求日志穿插;
- ▶ 格式随版本和组件持续变化;
- ▶ 性能退化未必产生明确 ERROR。

传统序列假设的风险

- ▶ 按文件顺序建模会混合不同实例;
- ▶ 只看单组件忽略跨服务依赖;
- ▶ 固定 Parser 容易被格式漂移破坏;
- ▶ 检测到异常后仍难定位具体实例。

SwissLog 的问题定义 利用日志中的标识关系，把交错的无结构日志恢复成与运行实例相关的序列，再进行异常检测与定位。

SwissLog 2/5: 先用 ID 关系图恢复 “谁和谁属于一起”

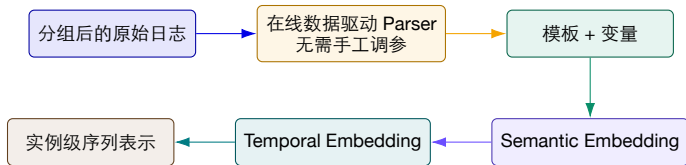


输入 日志中的 request、session、task、query 等标识符。

输出 跨组件 ID Relation Graph 和按运行实例分组的消息序列。

教学理解：ID 不是越多越好；关键是它能否稳定传播、唯一指代并建立跨组件关系。

SwissLog 3/5: Parser 既要适应格式漂移，也要保留语义与时间



语义 区分“连接池耗尽”与“连接池已恢复”，而不只看模板编号。

时间 性能问题可能表现为事件间隔拉长，而不是错误事件出现。

SwissLog 4/5: 检测与定位是两个不同问题

异常检测

- ▶ Attention-based Bi-LSTM 建模实例级日志序列；
- ▶ 同时利用事件语义和时间关系；
- ▶ 输出某个运行实例是否异常；
- ▶ 可覆盖显式错误与潜在性能问题。

异常定位

- ▶ 沿 ID 关系和异常贡献搜索；
- ▶ 缩小到相关组件或实例；
- ▶ 给出值班人员可继续验证的候选；
- ▶ 不能把模型注意力直接等同于因果根因。

教学区分 Detection 回答“这条实例轨迹是否异常”；Localization 回答“异常最可能在哪个实体或片段”。

SwissLog 5/5: 怎样把论文结论转化为工程判断?

论文报告	对工程的启示	上线前必须验证
跨组件 ID 关系 在线 Parser 语义 + 时间表示 实例级检测定位 真实与合成数据评估	日志 Schema 必须支持稳定关联 格式漂移不应要求频繁人工调参 性能异常不只依赖错误码 输出更贴近处置对象 机制具备实验支持	ID 覆盖率、冲突率、传播断点 新版本模板发现延迟与误合并 时钟漂移、采样与延迟分布变化 拓扑变化、未知故障与误报成本 本企业日志、语言和系统迁移性

结论边界 论文实验支持方法在其数据与设置下有效；生产采用还需要检查日志格式约定、ID 完整性、资源开销和跨版本鲁棒性。

代表性日志研究：从解析、压缩到异常检测

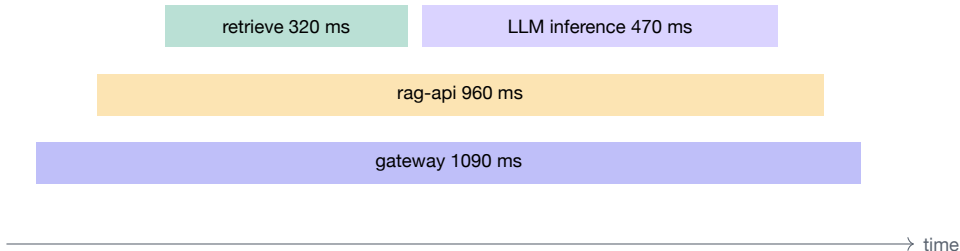
工作	研究问题	本课程中的位置
SLOPE (TOSEM 2026)	语法与 LLM 蒸馏语义结合的细粒度日志解析	Schema/Parser 如何兼顾结构与语义
LogBoost (TSC 2026)	选择高价值日志序列增强异常检测	数据质量比“全部输入”更重要
LogShrink (ICSE 2024)	利用日志共性与差异实现有效压缩	冷热分层与可查询压缩
LogReducer (ICSE 2023)	在线识别并减少内核日志热点	从源端控制日志成本
Prefix Caching for Log Parsing (ICSE 2026)	加速 LLM 在线日志解析	AI 方法也必须满足吞吐与成本约束

这些工作将在异常检测章节继续展开；本讲先建立它们依赖的日志结构、关联和成本问题。

三、分布式追踪：还原一次请求的因果路径

Trace 把跨进程的结构化事件组织成有父子、时间与因果关系的执行图。

Trace 最擅长回答：时间花在哪里，调用经过哪里？

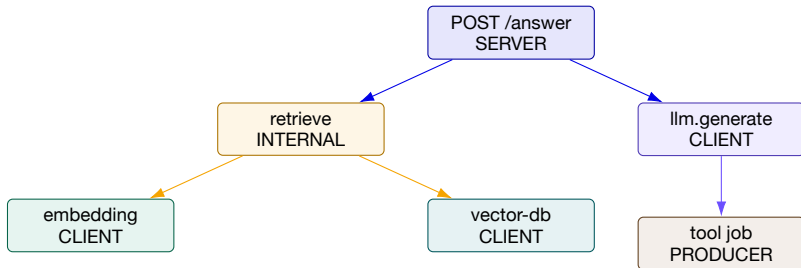


路径 请求经过哪些服务、数据库、队列和工具。

时间 每一步开始、结束、重叠、排队与等待。

依赖 父子、同步、异步和跨 Trace 的因果关系。

Trace 与 Span: 一棵树只是最常见的视图



注意 异步队列、批处理、消息扇入/扇出和缓存命中可能形成 DAG 或跨 Trace Link；不能强行用父子树表达所有因果关系。

关键路径不是“最慢 Span 排序”

例子

- ▶ 两个检索并行：A=300ms，B=420ms；
- ▶ LLM 必须等待两者完成后开始；
- ▶ 日志写入耗时 500ms，但在后台异步执行；
- ▶ 总请求耗时 900ms。

判断

- ▶ B 在关键路径上，A 的部分时间与 B 重叠；
- ▶ 异步日志虽然更慢，但不一定影响用户延迟；
- ▶ 应分析父子关系、重叠和等待，而非只看 duration；
- ▶ Self time 与 blocking time 需要区分。

优化纪律 只有缩短关键路径上的不可重叠时间，才会直接降低端到端延迟。

一个 Span 至少包含哪些信息？

<code>trace_id / span_id</code>	全局与局部身份	<code>attributes</code>	可查询的操作上下文
<code>parent_span_id</code>	父子关系	<code>events</code>	有时间戳的关键事件
<code>name / kind</code>	操作名与 Client/Server 等角色	<code>links</code>	跨 Trace/DAG 因果关联
<code>start / end</code>	时间范围	<code>status</code>	Unset / Error / Ok

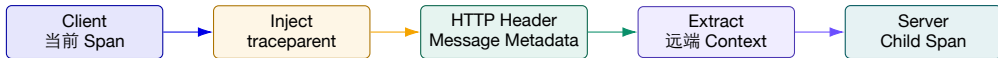
Resource 描述 “谁产生这个 Span”；Instrumentation Scope 描述 “由哪个库/版本产生”。

Attributes、Events、Links 和 Status 各有职责

元素	适合记录	例子
Attribute	操作期间成立、用于过滤或聚合的属性	model.name、db.system、batch.size
Span Event	某个具体时间点发生的事件	retry、cache.miss、exception
Span Link	非父子但存在因果关系的 Span	消息消费、批处理输入、Agent 子任务
Status	对 Span 操作结果的明确判断	Error；成功通常保持 Unset 即可

常见错误 把每个日志都建成 Span 会制造海量短 Span；把所有动态信息设为 Attribute 又会增加采样和索引成本。

上下文传播：Trace 能跨服务成立的核心机制

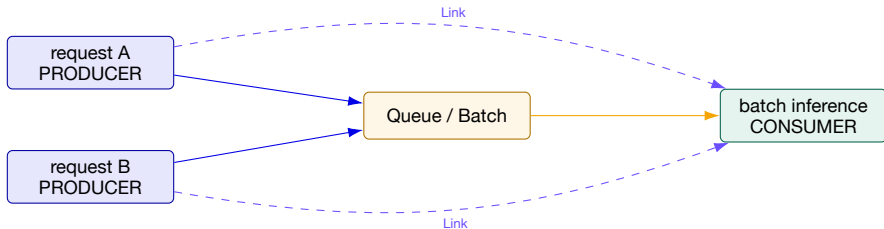


同步调用 HTTP/gRPC 客户端注入，服务端提取并创建 Server Span。

传播断点 代理删除 Header、异步任务未携带上下文、线程池切换丢失 Context。

Trace ID 相同并不能自动证明父子关系；Parent、Kind、时间和协议语义需要共同验证。

异步任务与批处理：使用 **Producer/Consumer** 和 **Span Links**



为什么不能只选一个 **Parent** 批处理 Span 同时由多个输入请求触发；Links 保留多对一因果关系，避免把其他请求错误挂到一个父节点下。

Baggage: 可以传播业务上下文，但不是免费的全局变量

适合

- ▶ tenant tier、experiment cohort;
- ▶ 需要跨服务传播且被下游埋点读取的有限信息;
- ▶ 不直接包含敏感原文;
- ▶ 大小、键数量和生命周期受控。

风险

- ▶ 每次跨服务都增加 Header/消息大小;
- ▶ 可能传播到不可信边界;
- ▶ 高基数字段进入 Span/日志后放大成本;
- ▶ 过期或错误值可能污染整个调用链。

原则 Baggage 负责传播，不自动意味着“必须记录”或“允许索引”；记录与访问需要单独策略。

自动埋点与手工埋点：覆盖边界不同

方式	擅长	盲点
自动埋点 手工 Span 零侵入/ eBPF 日志关联	HTTP/gRPC、数据库、常见框架；低改造成本 表达领域操作、关键阶段和业务结果 不修改代码观察网络、系统调用与运行时 复用现有事件语义和错误细节	不理解检索、 Prefill 、工具调用等业务阶段 容易命名不一致、漏异常路径、增加维护成本 语义恢复困难，异步/复用连接的因果关系复杂 没有传播和结构时难以重建完整路径

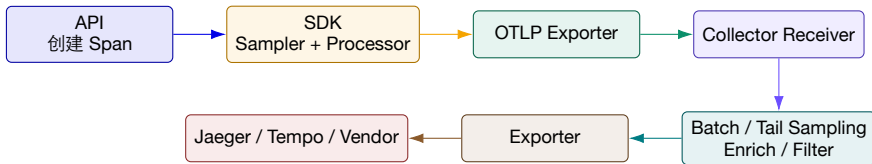
推荐组合 自动埋点覆盖边界，手工 **Span** 描述关键业务语义，日志记录详细事件，**eBPF** 补充内核与未埋点组件。

语义约定让跨团队 **Trace** 可以被同一种查询理解

<code>service.name</code>	稳定逻辑服务名	<code>gen_ai.request.model</code>	请求模型
<code>http.request.method</code>	HTTP 方法	<code>gen_ai.usage.input_tokens</code>	输入 Token
<code>server.address</code>	被调用目标	<code>error.type</code>	错误类型
<code>db.system</code>	数据库类型	<code>deployment.environment.name</code>	部署环境

治理点 禁止各团队分别使用 `model/modelName/llm_model`；语义字段需要版本、Owner 和兼容策略。

OpenTelemetry Trace 管道：API 产生语义，Collector 管理数据流



SDK 负责源端采样和导出；Collector 负责集中处理、路由、尾部采样和后端适配。

Python 示例：自动覆盖 HTTP，手工补充检索与推理阶段

```
tracer = trace.get_tracer("rag-api")
with tracer.start_as_current_span("rag.retrieve") as span:
    span.set_attribute("db.system", "milvus")
    span.set_attribute("rag.top_k", top_k)
    docs = retriever.search(query)
with tracer.start_as_current_span("gen_ai.generate") as span:
    span.set_attribute("gen_ai.request.model", model)
    answer = llm.generate(prompt)
    span.set_attribute("gen_ai.usage.output_tokens", answer.tokens)
```

应补充 异常记录、Status Error、重试
Event、缓存命中与业务结果。

不应记录 完整 Prompt、Secret、无限
枚举标签和原始用户身份。

版本提醒：优先使用 OTLP，而不是绑定旧 Jaeger Thrift Exporter

旧教程常见方式

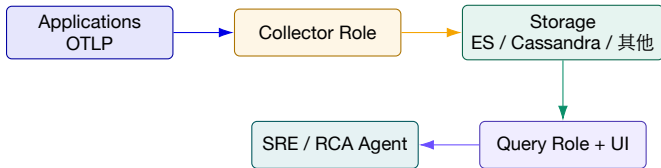
- ▶ SDK 直接配置 Jaeger Agent/Thrift exporter；
- ▶ 每种后端使用专用 exporter；
- ▶ 日志、指标和 Trace 使用不同代理；
- ▶ 迁移后端时修改应用配置。

当前推荐方式

- ▶ 应用统一通过 OTLP 发往 Collector 或 Jaeger v2；
- ▶ Collector 统一批处理、重试、采样、富化和路由；
- ▶ 后端替换尽量不影响应用；
- ▶ 直接导出仅适合简单开发环境。

迁移方法 保留旧示例用于理解组件，但实验和新项目使用 OTLP gRPC/HTTP。

Jaeger v2: 基于 OpenTelemetry Collector 框架的可配置追踪平台



角色 collector 接收并写存储; query 提供 API/UI; 可选 ingester 从 Kafka 写入。

部署 all-in-one 适合开发; 生产可拆分读写流量并水平扩展。

Collector 到存储：直接写入还是引入 Kafka 缓冲？

路径	优势	代价与风险
Collector → Storage	组件少、延迟低、易于理解	存储需承受峰值；持续拥塞可能丢数据
Collector → Kafka → Ingestor	持久缓冲、削峰、写入可独立扩展	组件与运维复杂度增加；积压会造成数据延迟
OTel Collector → Jaeger	可统一富化、过滤和其他信号采集	两级 Collector 需要避免重复处理与队列失控

必须监控追踪系统自身 接收速率、拒绝/丢弃 Span、队列长度、导出失败、存储写延迟和尾部采样等待内存。

为什么需要采样：不是所有正常请求都值得完整保存

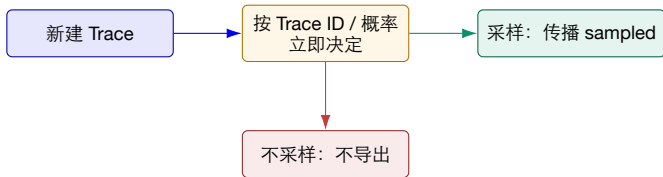
成本来源

- ▶ 应用创建与序列化 Span；
- ▶ 网络发送与 Collector 处理；
- ▶ 索引、对象存储与副本；
- ▶ 查询扫描与下游 RCA 计算；
- ▶ 隐私与访问控制范围扩大。

不能只问“采样率多少”

- ▶ 哪些错误和长尾必须保留？
- ▶ 低流量服务是否会被饿死？
- ▶ 新版本/新租户是否提高采样？
- ▶ 丢失一条罕见 Trace 的机会成本多大？
- ▶ 采样器自身是否可用、可解释？

头部采样：尽早决定，便宜但看不到最终结果



优点 源端减负、决策简单、确定性概率
采样可保持整条 Trace。

缺点 无法保证保留最终发生错误、超时
或罕见路径的请求。

尾部采样：看完整体再决定，信息更好但状态更重



可以依据 整体延迟、任一 Span Error、新版本、特定属性。

运行代价 有状态缓冲、路由一致性、内存与等待窗口。

失败模式 过载降级、未完成 Trace、规则漂移与低流量偏差。

采样失败的三种方式：偏、断、迟

偏 | Bias 只保留常见高流量服务，低流量或新路径缺少代表性。

断 | Broken Trace 不同服务决策不一致、Span 晚到或传播丢失，Trace 不完整。

迟 | Latency 尾部采样等待过久，事故发生后 Trace 迟迟不可查。



采样质量 不能只看最终保留比例，还要看异常覆盖、路径多样性、查询命中、Trace 完整性和下游诊断效果。

TraStrainer 1/3: 仅按 Trace 多样性采样为什么不够?

多样性视角

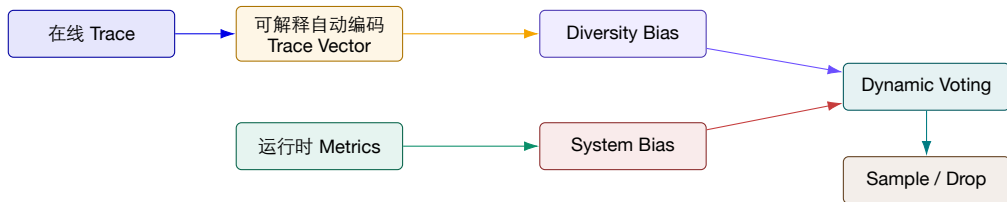
- ▶ 保留罕见路径、边缘调用模式;
- ▶ 减少大量重复正常 Trace;
- ▶ 适合描述行为覆盖度;
- ▶ 但未必知道系统当前是否正在故障。

运行状态视角

- ▶ CPU、内存、延迟、错误等指标反映异常窗口;
- ▶ 故障时常见路径也可能非常重要;
- ▶ 正常期与异常期应有不同采样偏好;
- ▶ 指标本身也可能噪声大或滞后。

论文主张 采样质量需要同时考虑 Trace diversity 与 system runtime state。

TraStrainer 2/3: 系统偏置与多样性偏置动态投票



教学理解 异常窗口提高“系统偏置”，稳定窗口强调“路径多样性”；投票机制在覆盖与故障相关性之间动态权衡。

TraStrainer 3/3: 评价采样器要看下游 RCA，而不只是压缩率

论文报告

- ▶ 在线采样器结合运行时状态与 Trace 多样性；
- ▶ 在两个数据集、四个基线的比较中评估；
- ▶ 下游 Top-1 RCA 准确率平均提升 32.63%；
- ▶ 说明“留下哪些 Trace”会改变诊断质量。

工程检查

- ▶ 指标异常是否总能代表故障？
- ▶ 新服务和低流量路径是否被公平覆盖？
- ▶ 投票和阈值变化是否可解释、可回放？
- ▶ 采样器过载时怎样降级？
- ▶ 是否在本企业 RCA 方法上仍有收益？

Mint 1/7: 传统“保留或丢弃”为什么造成诊断盲区?

整条保留或丢弃

- ▶ 采样 Trace: 完整保留;
- ▶ 未采样 Trace: 完全丢弃;
- ▶ 事后未知查询无法命中;
- ▶ 单条 Trace 内部的重复结构没有被压缩。

Mint 的生产观察

- ▶ 阿里某大规模系统每天产生 18.6–20.5 PB Trace;
- ▶ 研究期内, 传统采样导致约 27.17% 查询 miss;
- ▶ 要分析哪些请求通常无法事先完全预测;
- ▶ 成本问题既来自 Trace 数量, 也来自单条 Trace 体积。

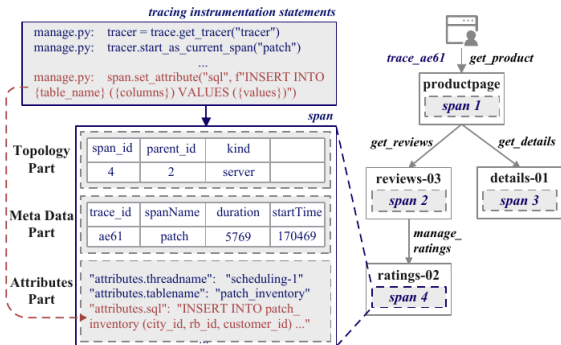
问题转化 能否为所有请求保留低成本“骨架”, 只为高价值请求保留完整变量?

Mint 2/7: Trace 同时包含拓扑、元数据与高变化属性

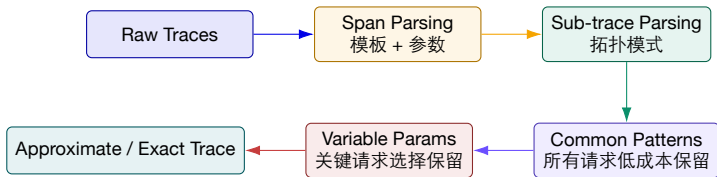
读图顺序

1. Topology: Span 与父子关系;
2. Metadata: Trace ID、操作名、时间;
3. Attributes: SQL、参数、线程等变量;
4. 多条 Trace 共享结构, 但变量不同。

压缩机会 结构和稳定字段形成 Commonality; 请求参数与运行时值形成 Variability。



Mint 3/7: 从“采样”转向“共性 + 差异”

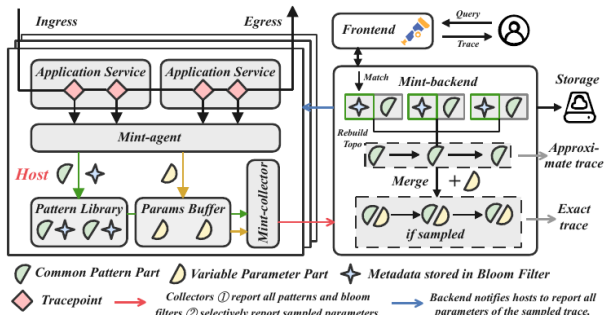


未采样请求 保留模式、拓扑和可检索元数据，返回近似 Trace。

采样请求 同时保留变量参数，可重建精确 Trace。

思想类似日志模板化，但 Trace 还必须保留跨 Span 的拓扑结构和跨节点一致性。

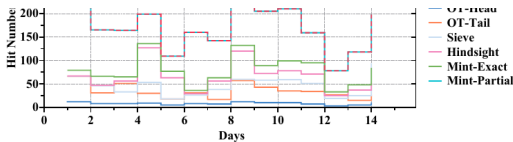
Mint 4/7: Agent 侧压缩同时减少网络与存储



核心组件

- ▶ Pattern Library: Span 与拓扑模式;
- ▶ Params Buffer: 暂存变量参数;
- ▶ Bloom Filter: 把 Trace 元数据挂到模式;
- ▶ Collector: 报告模式并选择参数;
- ▶ Backend: 合并并重建近似/精确 Trace。

Mint 5/7: 查询未采样请求时，至少返回“近似路径”

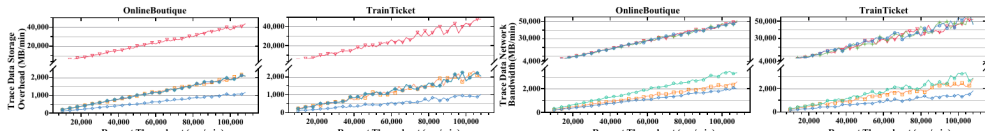


论文中的查询分类

- ▶ Exact hit: 返回完整参数和路径;
- ▶ Partial hit: 返回模式化近似 Trace;
- ▶ Miss: 没有任何记录;
- ▶ Mint 的 partial + exact 覆盖全部查询。

价值 近似 Trace 可能已经足够回答“经过哪些服务、在哪一步分叉、是否命中罕见路径”。

Mint 6/7: 论文报告的成本结果要和基线位置一起读



OT-Head 源端减少网络与存储，但未采样请求完全丢失。

OT-Tail 后端决定，存储可降，但数据已经过网络。

Mint Agent 侧解析压缩；平均存储降至 2.7%，网络降至 4.2%。

百分比相对于完整追踪基线；结果来自 OnlineBoutique、TrainTicket 与生产数据设置，不应直接外推到所有系统。

Mint 7/7: 保留结构信息能提升下游 RCA，但仍有边界

评价维度	论文证据	教学解释
查询可用性 下游 RCA	未采样请求可返回 approximate trace 在 MicroRank、TraceAnomaly、TraceRCA 上 A@1 更高	诊断不再从完全空白开始 采样策略会改变算法可见的因果结构
部署成本	生产运行两个月; Pattern 存储约为原始 Trace 的 小比例	共性稳定时收益大，频繁变更会增加重建
完整性	Bloom Filter 与跨 Agent 参数通知保持可检索与 一致	假阳性、晚到、节点故障仍需处理

最重要的思想 可观测数据压缩不应只追求字节最少，而要保留未来未知诊断问题所需的结构。

ZeroTracer 1/3: 无侵入 Trace 为什么容易不准确?

代码埋点的代价

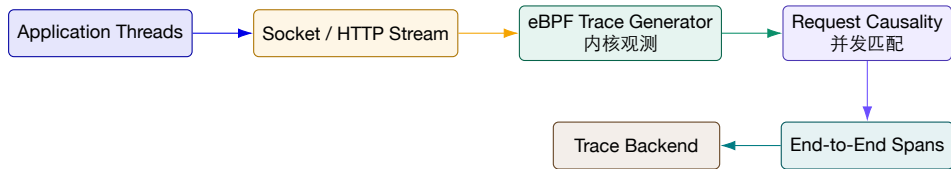
- ▶ 修改或注入框架与第三方库;
- ▶ 语言和协议覆盖不一致;
- ▶ 升级后探针可能失效;
- ▶ 多线程、异步与自定义网络栈仍可能丢上下文。

纯网络观测的困难

- ▶ 长连接上并发多个请求;
- ▶ 请求与响应的因果匹配复杂;
- ▶ TLS、缓冲和用户态框架隐藏边界;
- ▶ 只看到数据包, 不理解请求语义。

ZeroTracer 的目标 在内核中使用 eBPF 生成 HTTP 请求 Trace, 尽量做到零代码埋点, 同时恢复准确请求因果关系。

ZeroTracer 2/3: 在带内数据路径中关联请求与响应



优势 不依赖业务代码修改，适合遗留系统和第三方组件。

限制 主要面向 HTTP；加密、协议变化、极端并发和内核兼容仍需验证。

ZeroTracer 3/3: 准确率与开销必须一起读

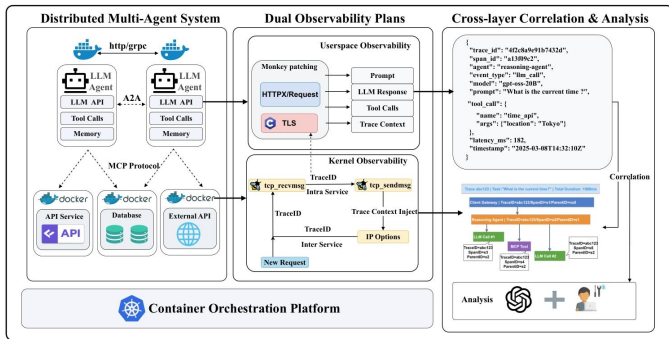
作者报告的结果

- ▶ Trace accuracy 超过 91%;
- ▶ 不同并发下性能相对稳定;
- ▶ 请求延迟增加约 0.5%–1.2%;
- ▶ CPU/内存消耗增加约 3%–5.8%;
- ▶ 支持多线程微服务请求追踪。

学生应检查

- ▶ accuracy 怎样定义, 错误匹配的代价是什么?
- ▶ 流量、协议、连接复用和硬件范围多大?
- ▶ 丢包、重传、TLS 和 Service Mesh 如何影响?
- ▶ 内核升级与 eBPF 安全策略是否兼容?
- ▶ 语义 Span 仍需怎样补充?

AgentTracer: 语义可观测与内核可观测是互补的两张图



教学读图

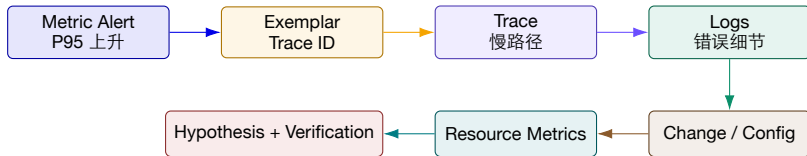
1. 用户态知道 Prompt、LLM、Tool 与 Agent 语义；
2. 内核态知道 socket、进程与真实通信；
3. TraceID 把两个平面对齐；
4. 联合分析区分“计划如此”与“实际执行”。

定位 这是项目阶段性研究材料，不包装为已正式发表论文。

四、跨信号关联与可观测数据治理

真正的诊断入口不是某个后端，而是对象、时间、请求与证据之间的统一关系。

跨信号诊断：从聚合症状逐步下钻到具体证据



注意 箭头表示调查顺序，不表示天然因果；每一步都应给出支持、反驳或缺失证据。

Exemplar: 在聚合指标上保留少量具体请求入口

```
http_server_duration_seconds_bucket  
{le="5",service="rag-api"} 18234  
# {trace_id="4f2c8a9e..."} 4.83
```

- ▶ Histogram 仍保持低基数;
- ▶ 示例请求携带 Trace ID;
- ▶ Dashboard 上点击异常点进入 Trace。

不要混淆

- ▶ Exemplar 不是把 Trace ID 变成指标标签;
- ▶ 它只保存少量代表样本;
- ▶ 采样策略影响点击后能否查到完整 Trace;
- ▶ 需要统一时间窗和后端链接配置。

统一时间与对象身份：关联正确性的两个地基

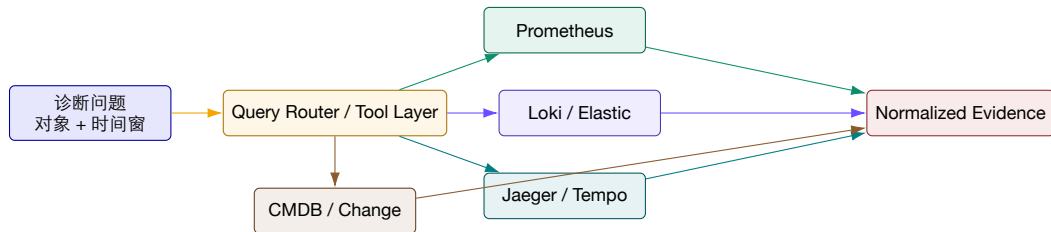
时间

- ▶ 区分 event time、observed time、ingest time；
- ▶ 使用 NTP/PTP 并监控 clock offset；
- ▶ 保留时区和高精度时间戳；
- ▶ 跨后端查询考虑延迟、乱序和迟到数据；
- ▶ “先发生”是因果判断的必要但非充分条件。

身份

- ▶ service.name 与部署对象建立稳定映射；
- ▶ Pod/容器是短生命周期实例；
- ▶ 版本、区域、集群、租户需要显式字段；
- ▶ CMDDB/拓扑负责对象关系；
- ▶ 变更事件必须绑定目标对象和版本。

统一查询不是把所有数据复制进一张“大宽表”



联邦式思路 保留各后端擅长的数据模型，通过统一身份、时间、权限、查询接口和证据格式实现关联。

TELLER: 跨层 Trace 与日志必须对齐到同一个请求

对齐步骤

1. 用 NVTX 范围重建引擎/算子层次;
2. 用 CUPTI correlation ID 绑定 Runtime、Driver 与 Kernel;
3. 将并发步骤投影到 request-level call-chain;
4. 按 request ID 和时间窗对齐日志;
5. 生成结构化 Trace-Log 诊断样本。

为什么只按时间不够

- ▶ 并发请求共享同一算子和设备时间段;
- ▶ 日志可能早于或晚于真实设备事件;
- ▶ Host 调用与 Kernel 需要 correlation 关系;
- ▶ 同一故障窗口内存在大量正常请求;
- ▶ 请求级投影减少错误归因。

多租户治理：可查询不等于可以被任何人或 Agent 读取

治理面	风险	控制
租户隔离 字段权限 工具调用 数据驻留 证据引用	跨租户日志、Trace 或 Prompt 泄露 运维人员不应读取完整业务内容 RCA Agent 扩大查询范围或绕过审批 日志跨区域或跨境复制 报告复制敏感原文	Tenant ID 强制注入、后端隔离与查询过滤 字段级脱敏、动态授权、审计 Tool Scope、最大时间窗、只读凭证 区域路由、存储策略、加密与删除证明 引用 ID、摘要、最小必要片段

安全默认值 诊断智能体应在选定集群、命名空间、对象和时间窗内工作；扩大范围需要显式理由和审计。

证据质量模型：支持、反驳、缺失与冲突

假设 H：向量库连接池耗尽导致慢请求

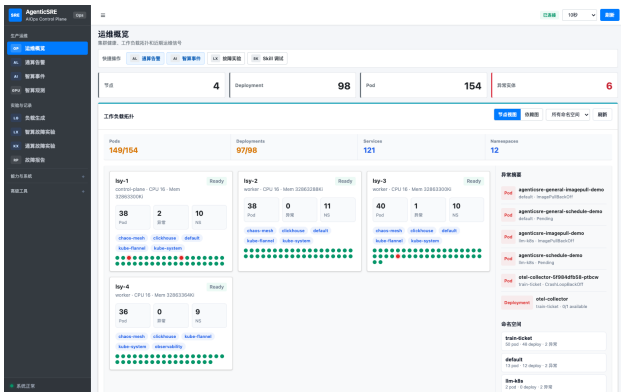
- ▶ 支持：慢 Trace 的 retrieve Span 占 82%；
- ▶ 支持：日志出现 PoolTimeout；
- ▶ 支持：连接池配置刚从 80 改为 20；
- ▶ 反驳：部分正常请求使用同一实例；
- ▶ 缺失：数据库端活跃连接指标中断。

诊断输出

- ▶ 结论与置信度；
- ▶ 支持和反驳证据引用；
- ▶ 未获取数据与失败工具；
- ▶ 替代假设：网络延迟或查询复杂度；
- ▶ 验证动作：恢复配置并观察连接与 P95。

纪律 自然语言解释必须可以回到具体查询、对象、时间窗和原始证据。

企业平台案例：界面入口应对真实对象与下钻路径



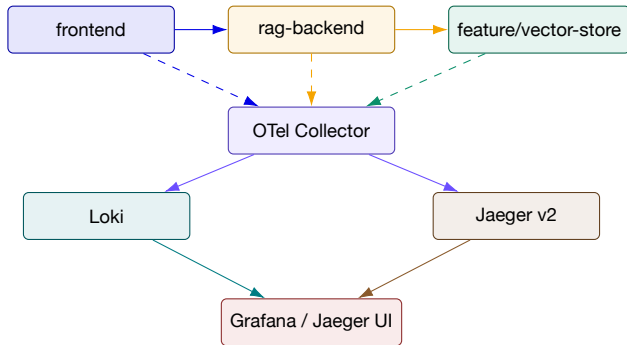
从 UI 检查后端能力

- ▶ 节点、Deployment、Pod 是否有稳定对象 ID；
- ▶ 异常实体是否能下钻指标、日志和 Trace；
- ▶ 查询是否继承集群/命名空间/时间窗；
- ▶ 故障报告是否保留证据与工具调用；
- ▶ 多集群上下文是否隔离。

五、实验：三服务推理栈的 日志—Trace关联

依次生成日志和 Trace、注入故障、关联查询、提出假设并检查恢复。

实验架构与交付物



交付：结构化日志、三服务 Trace、自定义 Span、日志跳转 Trace、一个慢调用诊断记录和数据治理说明。

步骤 1：输出稳定结构并自动注入 Trace 上下文

```
log.info("retrieval.completed", extra={
    "event.name": "retrieval.completed",
    "service.name": "rag-backend",
    "duration_ms": elapsed_ms,
    "result.count": len(docs),
    "db.system": "milvus"
})
```

要求 每条记录包含 timestamp、severity、service、event.name、trace_id、span_id。

验证 正常与异常路径字段一致；不记录 Prompt、API Key 和用户原文。

步骤 2: Collector 同时接收 OTLP 与容器日志

```
receivers:
  otlp: {protocols: {grpc: {}, http: {}}}
  filelog:
    include: [/var/log/containers/*.log]
processors:
  memory_limiter: {limit_mib: 512}
  k8sattributes: {}
  batch: {}
exporters:
  otlp/jaeger: {endpoint: jaeger:4317, tls: {insecure: true}}
  otlphttp/loki: {endpoint: http://loki:3100/otlp}
service:
  pipelines:
    traces: {receivers: [otlp], processors: [memory_limiter,batch],
exporters: [otlp/jaeger]}
    logs: {receivers: [otlp,filelog], processors:
[memory_limiter,k8sattributes,batch], exporters: [otlphttp/loki]}
```

检查点

- ▶ Collector 自身队列与拒绝指标可见；
- ▶ Resource 属性补充集群/命名空间；
- ▶ trace_id 保留为元数据而非 Loki 标签；
- ▶ 生产环境配置认证、TLS 与持久队列；
- ▶ 失败导出不能无限重试阻塞。

步骤 3: 自动传播 + 自定义检索和推理 Span

```
FastAPIInstrumentor.instrument_app(app)
HTTPXClientInstrumentor().instrument()

with tracer.start_as_current_span("rag.retrieve") as span:
    span.set_attribute("rag.top_k", 5)
    docs = client.get("http://vector-store/search").json()

with tracer.start_as_current_span("gen_ai.generate"):
    answer = generate(docs)
```

验证传播 三个服务 Span 共享 Trace ID, Server Span 的 Parent 对应上游 Client Span。

验证语义 Trace 中能分别看到 HTTP、retrieve 和 generate, 而不只有框架 Span。

步骤 4：从日志和 **Trace** 两个方向完成关联查询

日志 → Trace

```
{service_name="rag-backend"}  
| json  
| severity_text="ERROR"  
| trace_id!=""
```

- ▶ 点击 Derived Field 打开 Jaeger;
- ▶ 检查该请求所有 Span 与错误位置。

Trace → Logs

- ▶ 在慢 Trace 中选中 rag.retrieve;
- ▶ 使用 trace_id、service.name 和 Span 时间窗查询日志;
- ▶ 检查异常前后的重试、连接池与依赖事件;
- ▶ 记录缺失或冲突证据。

步骤 5：注入 500ms 延迟并完成可验证诊断

故障注入

- ▶ 在 vector-store 增加 500ms sleep;
- ▶ 只对 20% 请求或某个 tenant 生效;
- ▶ 同时产生结构化 `fault.injected` 事件;
- ▶ 保留注入开始、结束和目标对象。

诊断输出

1. 症状：端到端 P95 上升;
2. 路径：rag.retrieve 占据关键路径;
3. 日志：目标请求出现注入事件;
4. 假设：vector-store 延迟;
5. 验证：移除 sleep 后 P95 与 Span 恢复。

关键 必须能从最终结论回到具体 Trace、日志行、对象和注入记录，而不是只提交截图。

实验验收与常见故障

验收项	通过条件	常见故障
Trace 完整 日志结构 日志— Trace 关联 慢调用诊断 Collector 健康 治理说明	单请求覆盖三个服务与自定义 Span 必填字段完整、类型稳定、无敏感原文 双向点击/查询可到达同一请求 能指出关键路径和具体依赖 无持续丢弃、队列可控、导出成功 标签、保留、脱敏与租户边界清晰	Header 未传播、Parent 丢失 JSON 合法但 Schema 漂移 trace_id 被错误设为 Loki 标签 只按最长 Span 排序 内存限制、后端不可达、批处理延迟 把实验默认配置直接当生产方案

建议两人一组：一人负责埋点与数据管道，一人负责故障注入与诊断；最后交换角色复核。

时钟误差算例：日志看起来倒序，是否表示调用链错误？

设每台主机的时钟误差范围为 $\pm 80\text{ ms}$ ；日志时间已经换算到同一时区，但尚未消除偏移。

记录	显示时间	真实时间可能区间	可确认的信息
父 Span 发出	10:00:00.100	.020-.180	请求头携带父上下文
子 Span 接收	10:00:00.060	前一秒.980-本秒.140	接收后提取父上下文
表面差值	子比父早 40 ms	误差差值可达 160 ms	时间顺序不具判别力

排查顺序：验证上下文注入/提取，再看同机单调计时，最后校正跨机时间。

推导与判断 父子上下文支持调用关系；这组墙钟时间不能证明“子调用早于父调用”，也不能据此算出负网络延迟。

算例使用假设数据。

机制依据： [OpenTelemetry: Context propagation](#)；核对日期：2026-09-09。

参考资料与延伸阅读

- ▶ OpenTelemetry Documentation: Logs、Traces、Context Propagation、Sampling、Collector;
- ▶ Jaeger v2 Documentation: Architecture、Deployment、Storage;
- ▶ Grafana Loki Documentation: Labels、OTLP Ingestion、Promtail EOL、LogQL;
- ▶ Elastic Common Schema 9.5.0 与 Elasticsearch Data Streams / Lifecycle;
- ▶ Li et al., *SwissLog*, IEEE TDSC, 2023;
- ▶ Huang et al., *TraStrainer*, FSE, 2024;
- ▶ Huang et al., *Mint*, ASPLOS, 2025;
- ▶ Yang et al., *ZeroTracer*, IEEE TPDS, 2025;
- ▶ Li et al., *TELLER*, ASE, 2026;
- ▶ 本地材料: `ai-infra-engineer-learning-main/mod-108-monitoring-observability`;
- ▶ AgentTracer、AgenticSRE 系统原型材料。

官方链接: [OpenTelemetry Logs](#); [Sampling](#); [Jaeger Architecture](#); [Loki Labels](#)

让每个结论都能回到证据

让每条证据都保留语义、路径与边界

校企联合研究生课程 面向 AI 原生系统的智能运维